1)Write a Java program to demonstrate **exception handling using try and catch** by handling division by zero.

Input:
10 0

DivisionExample - Notepad

File Edit Format View Help

```
public class DivisionExample {
    public static void main(String[] args) {
        int a = 10;
        int b = 0;

        try {
            int result = a / b;
            System.out.println("Result: " + result);
        } catch (ArithmeticException e) {
            System.out.println("Cannot divide by zero.
        }
    }
}
```

Output:
Cannot divide by zero.

Select Command Prompt

```
Microsoft Windows [Version 10.0.14393]
(c) 2016 Microsoft Corporation. All rights reserved.

C:\Users\ITLAB2>cd Desktop

C:\Users\ITLAB2\Desktop>cd 25EU02067

C:\Users\ITLAB2\Desktop\25EU02067>javac DivisionExample.java

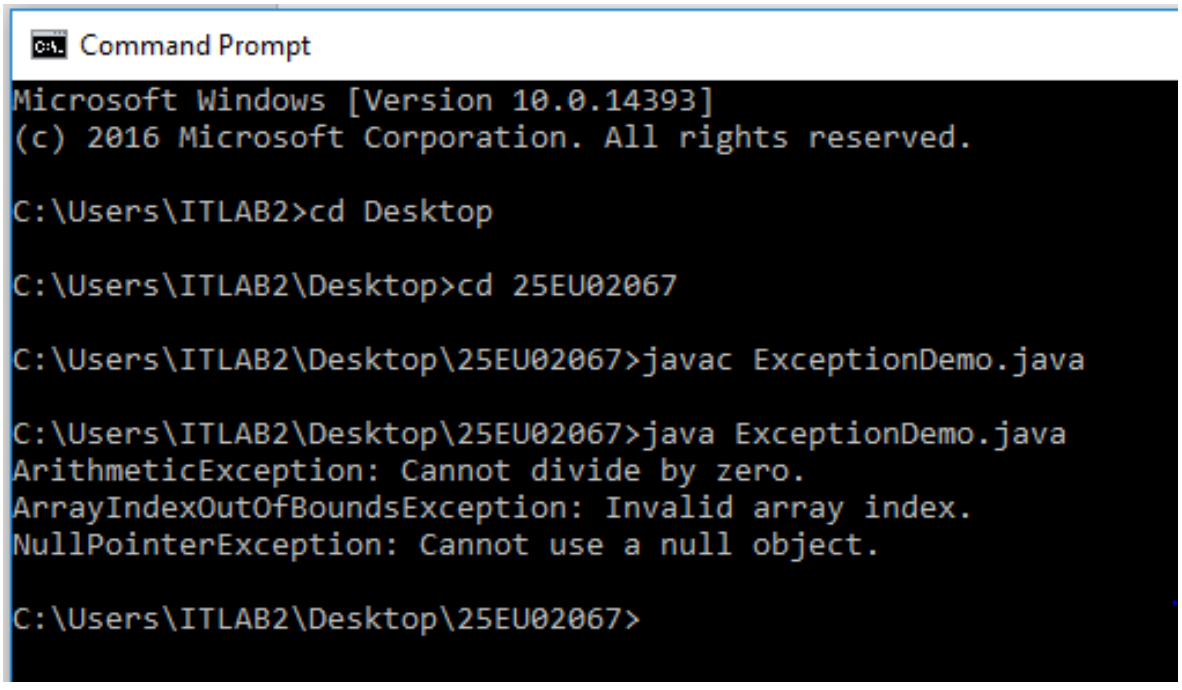
C:\Users\ITLAB2\Desktop\25EU02067>java DivisionExample.java
Exception caught: Cannot divide by zero.

C:\Users\ITLAB2\Desktop\25EU02067>
```

2)Write a Java program to demonstrate different **types of exceptions** such as `ArithmeticException`, `ArrayIndexOutOfBoundsException`, and `NullPointerException`.
Output: ArithmeticException handled. ArrayIndexOutOfBoundsException handled. NullPointerException handled.

Input:
No user input required.

```
public class ExceptionDemo {
    public static void main(String[] args) {
        try {
            int a = 10;
            int b = 0;
            int result = a / b;
            System.out.println(result);
        } catch (ArithmeticException e) {
            System.out.println("ArithmeticException: Cannot divide by zero.");
        }
        try {
            int[] numbers = {10, 20, 30};
            System.out.println(numbers[5]);
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("ArrayIndexOutOfBoundsException: Invalid array index
        }
        try {
            String str = null;
            System.out.println(str.length());
        } catch (NullPointerException e) {
            System.out.println("NullPointerException: Cannot use a null object.");
        }
    }
}
```



3)Write a Java program to demonstrate an **uncaught exception** and observe the default JVM exception message and stack trace.
Output: Exception in thread "main" java.lang.ArithmeticException: / by zero, followed by the JVM stack trace.

Input:
No user input required.

```
public class UncaughtExceptionDemo {
    public static void main(String[] args) {
        System.out.println("Program started.");
        int a = 10;
        int b = 0;
        int result = a / b;
        System.out.println("Result: " + result);
    }
}
```

```
C:\Users\ITLAB2\Desktop\25EU02067>javac UncaughtExceptionDemo.java

C:\Users\ITLAB2\Desktop\25EU02067>java UncaughtExceptionDemo.java
Program started.
Exception in thread "main" java.lang.ArithmeticException: / by zero
        at UncaughtExceptionDemo.main(UncaughtExceptionDemo.java:6)

C:\Users\ITLAB2\Desktop\25EU02067>
```

4)Write a Java program to handle **multiple exceptions using multiple catch clauses**.
Output: ArithmeticException caught: cannot divide by zero.
Input: 10 0

```java
public class MultipleCatchDemo {
    public static void main(String[] args) {
        try {
            int[] numbers = {10, 20, 30};
            int a = 10;
            int b = 0;
            System.out.println("Result: " + (a / b));
            System.out.println(numbers[5]);
        } catch (ArithmeticException e) {
            System.out.println("Arithmetic Exception: Cannot divide by zero.");
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Array Index Exception: Array index is out of bounds.");
        } catch (Exception e) {
            System.out.println("General Exception: " + e.getMessage());
        }
        System.out.println("Program continues after exception handling.");
    }
}
```

```
C:\Users\ITLAB2\Desktop\25EU02067>javac MultipleCatchDemo.java

C:\Users\ITLAB2\Desktop\25EU02067>java MultipleCatchDemo.java
Arithmetic Exception: Cannot divide by zero.
Program continues after exception handling.

C:\Users\ITLAB2\Desktop\25EU02067>
```

5)Write a Java program to demonstrate the use of the **throw statement** to explicitly generate an exception when a given condition is violated.
Output: IllegalArgumentException: Number must be positive.
Input: -5

```java
public class ThrowExample {
    public static void main(String[] args) {
        int age = 15;
        try {
            if (age < 18) {
                throw new ArithmeticException("Age must be 18 or above.");
            }
            System.out.println("Eligible to vote.");
        }
        catch (ArithmeticException e) {
            System.out.println("Exception: " + e.getMessage());
        }
    }
}
```

```
C:\Users\ITLAB2\Desktop\25EU02067>javac ThrowExample.java

C:\Users\ITLAB2\Desktop\25EU02067>java ThrowExample.java
Exception: Age must be 18 or above.

C:\Users\ITLAB2\Desktop\25EU02067>
```

6)Write a Java program to demonstrate the use of the **throws keyword** by propagating an exception to the calling method.
Output: FileNotFoundException propagated to the calling method.
Input: missing.txt

```
public class ThrowsExample {
    static void divide() throws ArithmeticException {
        int a = 10;
        int b = 0;
        int result = a / b;
        System.out.println("Result: " + result);
    }
    public static void main(String[] args) {
        try {
            divide();
        }
        catch (ArithmeticException e) {
            System.out.println("Exception caught: " + e.getMessage());
        }
    }
}
```

```
C:\Users\ITLAB2\Desktop\25EU02067>javac ThrowsExample.java

C:\Users\ITLAB2\Desktop\25EU02067>java ThrowsExample.java
Exception caught: / by zero

C:\Users\ITLAB2\Desktop\25EU02067>
```

7)Write a Java program to demonstrate the use of the **finally block** for executing statements regardless of whether an exception occurs.
Output: Exception handled: cannot divide by zero. Finally block executed.
Input: 10 0

```
public class FinallyExample {
    public static void main(String[] args) {
        try {
            int a = 10;
            int b = 0;
            int result = a / b;
            System.out.println("Result: " + result);
        }
        catch (ArithmeticException e) {
            System.out.println("Exception caught: " + e.getMessage());
        }
        finally {
            System.out.println("Finally block is always executed.");
        }
    }
}
```

```
C:\Users\ITLAB2\Desktop\25EU02067>javac FinallyExample.java

C:\Users\ITLAB2\Desktop\25EU02067>java FinallyExample.java
Exception caught: / by zero
Finally block is always executed.

C:\Users\ITLAB2\Desktop\25EU02067>
```

8)Write a Java program to create a **user-defined exception** by extending the `Exception` class.
Output: CustomException: Age must be 18 or above.
Input: 16

```
class AgeException extends Exception {
    AgeException(String message) {
        super(message);
    }
}
public class UserDefinedException {
    public static void main(String[] args) {
        int age = 15;
        try {
            if (age < 18) {
                throw new AgeException("Age must be 18 or above.");
            }
            System.out.println("Eligible to vote.");
        }
        catch (AgeException e) {
            System.out.println("Exception: " + e.getMessage());
        }
    }
}
```

```
C:\Users\ITLAB2\Desktop\25EU02067>javac UserDefinedException.java

C:\Users\ITLAB2\Desktop\25EU02067>java UserDefinedException.java
Exception: Age must be 18 or above.

C:\Users\ITLAB2\Desktop\25EU02067>
```

9)Write a Java program to validate a student's marks and throw a custom exception when the marks are outside the valid range.
Output: InvalidMarksException: Marks must be between 0 and 100.
Input: 105

```
class InvalidMarksException extends Exception {
    InvalidMarksException(String message) {
        super(message);
    }
}
public class StudentMarks {
    public static void validateMarks(int marks) throws InvalidMarksException {
        if (marks < 0 || marks > 100) {
            throw new InvalidMarksException("Marks must be between 0 and 100.");
        }
        System.out.println("Valid marks: " + marks);
    }
    public static void main(String[] args) {
        int marks = 105;
        try {
            validateMarks(marks);
        }
        catch (InvalidMarksException e) {
            System.out.println("Exception: " + e.getMessage());
        }
    }
}
```

```
C:\Users\ITLAB2\Desktop\25EU02067>java StudentMarks.java
Exception: Marks must be between 0 and 100.

C:\Users\ITLAB2\Desktop\25EU02067>
```

10)Write a Java program that combines **try, multiple catch, throw, throws and finally** in a single application.
Output: ArithmeticException caught: cannot divide by zero. Finally block executed.
Input: 10 0

```
class ExceptionDemo {
    static void checkMarks(int marks) throws Exception {
        if (marks < 0 || marks > 100) {
            throw new Exception("Marks must be between 0 and 100.");
        }
        System.out.println("Valid Marks: " + marks);
    }
    public static void main(String[] args) {
        int marks = 105;
        try {
            checkMarks(marks);
            int result = 10 / 0;
            System.out.println(result);
        }
        catch (ArithmeticException e) {
            System.out.println("Arithmetic Exception: " + e.getMessage());
        }
        catch (Exception e) {
            System.out.println("Exception: " + e.getMessage());
        }
        finally {
            System.out.println("Finally block executed.");
        }
    }
}
```

```
C:\Users\ITLAB2\Desktop\25EU02067>javac ExceptionDemo.java

C:\Users\ITLAB2\Desktop\25EU02067>java ExceptionDemo.java
Exception: Marks must be between 0 and 100.
Finally block executed.

C:\Users\ITLAB2\Desktop\25EU02067>
```

1)Write a Java program to demonstrate the use of **InputStream and OutputStream** to read bytes from an input source and write bytes to an output destination.
Output: Hello Java
Input: Hello Java

```java
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.IOException;
import java.io.InputStream;
import java.io.OutputStream;

public class StreamByteDemo {
    public static void main(String[] args) {
        String sourceFile = "source.txt";
        String destinationFile = "destination.txt";
        prepareSourceFile(sourceFile, "Hello, World! Demonstrating InputStream and OutputStream in Java.");
        try (InputStream in = new FileInputStream(sourceFile);
             OutputStream out = new FileOutputStream(destinationFile)) {
            byte[] buffer = new byte[1024];
            int bytesRead;

            while ((bytesRead = in.read(buffer)) != -1) {
                out.write(buffer, 0, bytesRead);
            }

            out.flush();

            System.out.println("Data successfully transferred from " + sourceFile + " to " + destinationFile);

        } catch (IOException e) {
            System.err.println("An I/O error occurred: " + e.getMessage());
        }
    }
    private static void prepareSourceFile(String filePath, String content) {
        try (OutputStream out = new FileOutputStream(filePath)) {
            byte[] dataBytes = content.getBytes();
            out.write(dataBytes);
        } catch (IOException e) {
            System.err.println("Failed to create source file: " + e.getMessage());
        }
    }
}
```

```
Microsoft Windows [Version 10.0.26200.9550]
(c) Microsoft Corporation. All rights reserved.

C:\Users\trivi>cd Javalab

C:\Users\trivi\Javalab>javac StreamByteDemo.java

C:\Users\trivi\Javalab>java StreamByteDemo.java
Data successfully transferred from source.txt to destination.txt

C:\Users\trivi\Javalab>
```

2)Write a Java program to **read the contents of a file byte-by-byte** using `FileInputStream` and display the contents on the console.
Output: Hello Java
Input: input.txt containing "Hello Java"

```java
import java.io.FileInputStream;
import java.io.FileNotFoundException;
import java.io.FileOutputStream;
import java.io.IOException;

public class ReadByteByByteDemo {
    public static void main(String[] args) {
        String fileName = "sample.txt";
        createSampleFile(fileName, "Hello! This file is being read byte-by-byte.");


        try (FileInputStream fis = new FileInputStream(fileName)) {
            int byteData;

            System.out.println("Reading contents of " + fileName + ":\n");

            while ((byteData = fis.read()) != -1) {

                System.out.print((char) byteData);
            }

            System.out.println("\n\n--- End of file reached ---");

        } catch (FileNotFoundException e) {
            System.err.println("File not found: " + e.getMessage());
        } catch (IOException e) {
            System.err.println("I/O error occurred while reading: " + e.getMessage());
        }
    }

    private static void createSampleFile(String fileName, String text) {
        try (FileOutputStream fos = new FileOutputStream(fileName)) {
            fos.write(text.getBytes());
        } catch (IOException e) {
            System.err.println("Could not create sample file: " + e.getMessage());
        }
    }
}
```

```
C:\Users\trivi\Javalab>javac ReadByteByByteDemo.java

C:\Users\trivi\Javalab>java ReadByteByByteDemo.java
Reading contents of sample.txt:

Hello! This file is being read byte-by-byte.

--- End of file reached ---

C:\Users\trivi\Javalab>
```

3)Write a Java program to **write data into a file** using `FileOutputStream` and verify the

contents of the file.
Output: Data written successfully. File contents: Java I/O Programming
Input: Java I/O Programming

```java
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.IOException;
import java.nio.charset.StandardCharsets;
public class FileOutputStreamVerifyDemo {
    public static void main(String[] args) {
        String filePath = "output.txt";
        String originalContent = "Welcome to Java I/O streams!\nWriting bytes and verifying content.";
        writeToFile(filePath, originalContent);
        verifyFileContent(filePath, originalContent);
    }
    private static void writeToFile(String filePath, String content) {
        try (FileOutputStream fos = new FileOutputStream(filePath)) {
            byte[] bytes = content.getBytes(StandardCharsets.UTF_8);
            fos.write(bytes);
            fos.flush();
            System.out.println("Data successfully written to '" + filePath + "'.");
        } catch (IOException e) {
            System.err.println("Error while writing to file: " + e.getMessage());
        }
    }
    private static void verifyFileContent(String filePath, String expectedContent) {
        try (FileInputStream fis = new FileInputStream(filePath)) {
            byte[] readBytes = fis.readAllBytes();
            String actualContent = new String(readBytes, StandardCharsets.UTF_8);
            System.out.println("\n--- Read Content from File ---");
            System.out.println(actualContent);
            System.out.println("------------------------------");
            if (expectedContent.equals(actualContent)) {
                System.out.println("Verification Result: SUCCESS (File contents match exactly).");
            } else {
                System.out.println("Verification Result: FAILED (Contents do not match).");
            }
        } catch (IOException e) {
            System.err.println("Error while reading file for verification: " + e.getMessage());
        }
    }
}
```

```
C:\Users\trivi\Javalab>javac FileOutputStreamVerifyDemo.java

C:\Users\trivi\Javalab>java FileOutputStreamVerifyDemo.java
Data successfully written to 'output.txt'.

--- Read Content from File ---
Welcome to Java I/O streams!
Writing bytes and verifying content.
------------------------------
Verification Result: SUCCESS (File contents match exactly).

C:\Users\trivi\Javalab>
```

4)Write a Java program to **copy the contents of one file to another file** using `FileInputStream` and `FileOutputStream`.
Output: File copied successfully to destination.txt.
Input: source.txt containing "File copy example"

```
import java.io.File;
import java.io.FileInputStream;
import java.io.FileNotFoundException;
import java.io.FileOutputStream;
import java.io.IOException;
public class FileCopyDemo {
    public static void main(String[] args) {
        String sourceFilePath = "source.txt";
        String destinationFilePath = "destination.txt";
        prepareDummyFile(sourceFilePath, "This is the source file content to be copied.\nJava File Streams Copy Demo.");
        copyFile(sourceFilePath, destinationFilePath);
    }
    public static void copyFile(String sourcePath, String destinationPath) {
        File src = new File(sourcePath);
        File dest = new File(destinationPath);
        if (!src.exists() || !src.isFile()) {
            System.err.println("Error: Source file does not exist or is not a regular file.");
            return;
        }
        try (FileInputStream fis = new FileInputStream(src);
             FileOutputStream fos = new FileOutputStream(dest)) {
            byte[] buffer = new byte[4096]; // 4 KB buffer for efficient I/O
            int bytesRead;
            long totalBytes = 0;
            while ((bytesRead = fis.read(buffer)) != -1) {
                // Write only the bytes that were actually read in this chunk
                fos.write(buffer, 0, bytesRead);
                totalBytes += bytesRead;
            }
            fos.flush(); // Ensure all buffered bytes are committed to disk
            System.out.println("File copied successfully!");
            System.out.println("Source: " + src.getAbsolutePath());
            System.out.println("Destination: " + dest.getAbsolutePath());
            System.out.println("Total bytes transferred: " + totalBytes);
        } catch (FileNotFoundException e) {
            System.err.println("File not found error: " + e.getMessage());
        } catch (IOException e) {
            System.err.println("I/O error during copy: " + e.getMessage());
        }
    }
    private static void prepareDummyFile(String path, String data) {
        try (FileOutputStream fos = new FileOutputStream(path)) {
            fos.write(data.getBytes());
        } catch (IOException e) {
            System.err.println("Could not create source file: " + e.getMessage());
        }
    }
}
```

```
C:\Users\trivi\Javalab>javac FileCopyDemo.java

C:\Users\trivi\Javalab>java FileCopyDemo.java
File copied successfully!
Source: C:\Users\trivi\Javalab\source.txt
Destination: C:\Users\trivi\Javalab\destination.txt
Total bytes transferred: 74

C:\Users\trivi\Javalab>
```

5) inpWrite a Java program to **copy an image from one file to another** using byte streams and handle `FileNotFoundException` and `IOException` using `try-catch-finally`.
Output: Image copied successfully to destination.jpg. Streams closed in the finally block.
Input: source.jpg

```
import java.io.File;
import java.io.FileInputStream;
import java.io.FileNotFoundException;
import java.io.FileOutputStream;
import java.io.IOException;
public class ImageCopyDemo {
    public static void main(String[] args) {
        String sourceImagePath = "input_image.jpg";
        String destinationImagePath = "copied_image.jpg";
        copyImage(sourceImagePath, destinationImagePath);
    }
    public static void copyImage(String sourcePath, String destinationPath) {
        FileInputStream fis = null;
        FileOutputStream fos = null;
        try {
            File sourceFile = new File(sourcePath);
            File destinationFile = new File(destinationPath);
            fis = new FileInputStream(sourceFile);
            fos = new FileOutputStream(destinationFile);
            byte[] buffer = new byte[4096];
            int bytesRead;
            long totalBytes = 0;
            while ((bytesRead = fis.read(buffer)) != -1) {
                fos.write(buffer, 0, bytesRead);
                totalBytes += bytesRead;
            }
            fos.flush();
            System.out.println("Image copied successfully!");
            System.out.println("Total bytes transferred: " + totalBytes);
        } catch (FileNotFoundException e) {
            System.err.println("File Error: The specified image file could not be found or opened.");
            System.err.println("Details: " + e.getMessage());
        } catch (IOException e) {
            System.err.println("I/O Error: An error occurred while reading or writing image data.");
            System.err.println("Details: " + e.getMessage());
        } finally {
            if (fis != null) {
                try {
                    fis.close();
                } catch (IOException e) {
                    System.err.println("Failed to close FileInputStream: " + e.getMessage());
                }
            }
            if (fos != null) {
                try {
                    fos.close();
                } catch (IOException e) {
                    System.err.println("Failed to close FileOutputStream: " + e.getMessage());
                }
            }
        }
    }
}
```